Library Management System: A Comprehensive Design

An integrated solution for efficient library operations — catalogue, circulation, user management, and reporting. This deck covers requirements, architecture, diagrams, estimation, schedule and testing for a robust LMS suitable for academic or public libraries.

LIBRARY LMS

SYSTEM DESIGN

Made with GAMMA

Problem Statement

The library faces manual cataloguing, slow circulation, inaccurate availability, and fragmented user records. Goals: automate circulation, centralise catalogue, enable search, manage acquisitions, and provide analytics while ensuring data integrity and user privacy.

Primary Users

Librarians, Members (students/public), Administrators

Key Constraints

Budget limits, legacy data migration, offline availability, security & privacy

Acceptance Criteria

Search <1s, 99% uptime, accurate fines calculation, role-based access

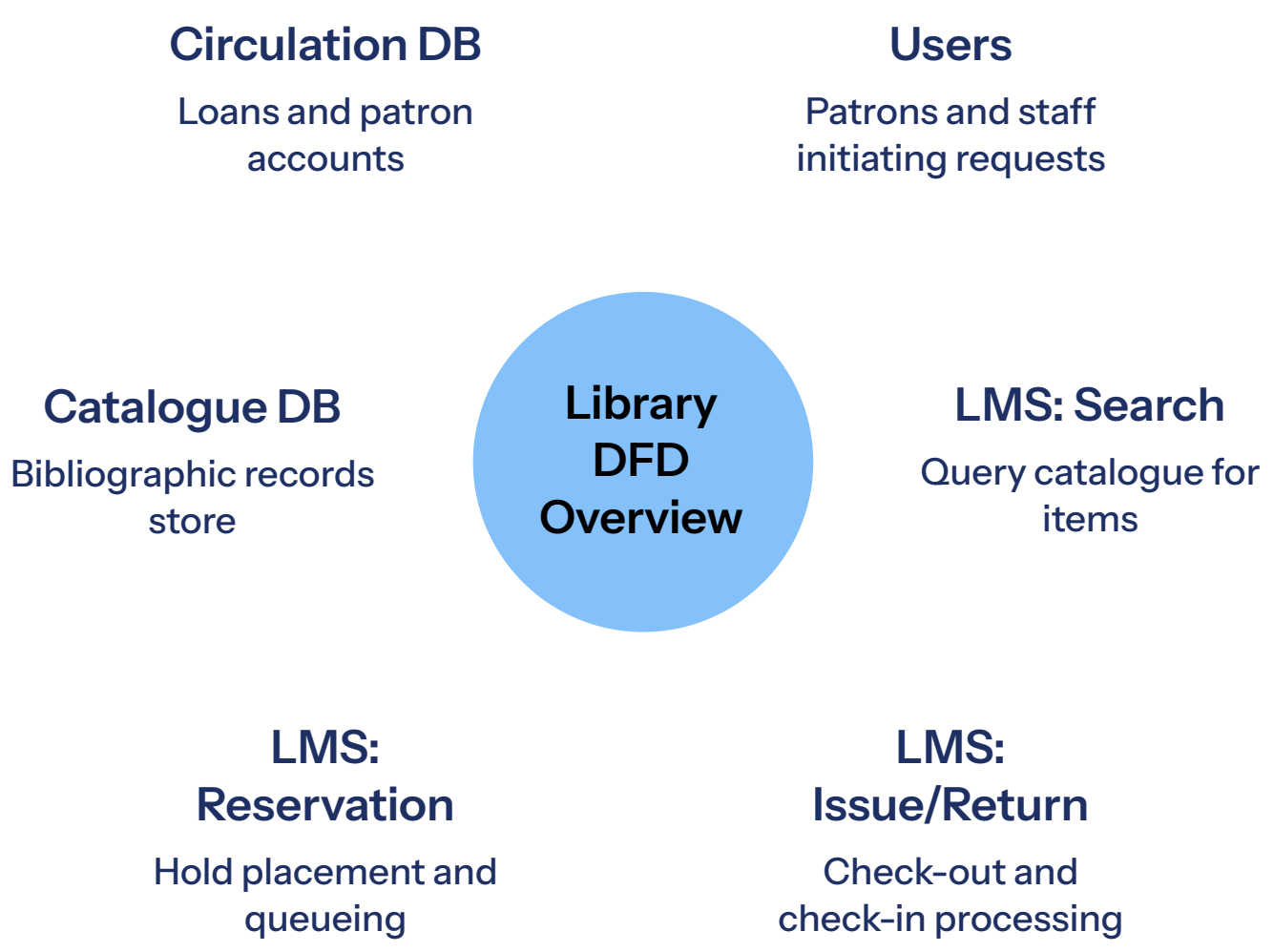
SRS Summary

- Functional: Catalogue CRUD, Issue/Return, Fine computation, Reservation, Notifications
- Non-functional: Security (RBAC), Performance, Backup, Scalability

Made with GAMMA

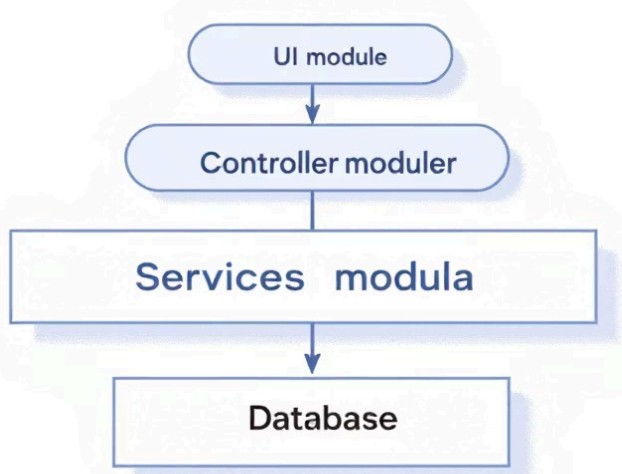
DFD — High Level (Level 0 & 1)

Top-level processes show data flow between Users, LMS, Catalogue, Circulation and External Services (Email/SMS). Level 1 expands issue/return, search, and reporting subprocesses.



Caption: The DFD clarifies how data moves: member requests trigger catalogue queries, circulation updates, and notifications. Use this to identify integration points and data stores.

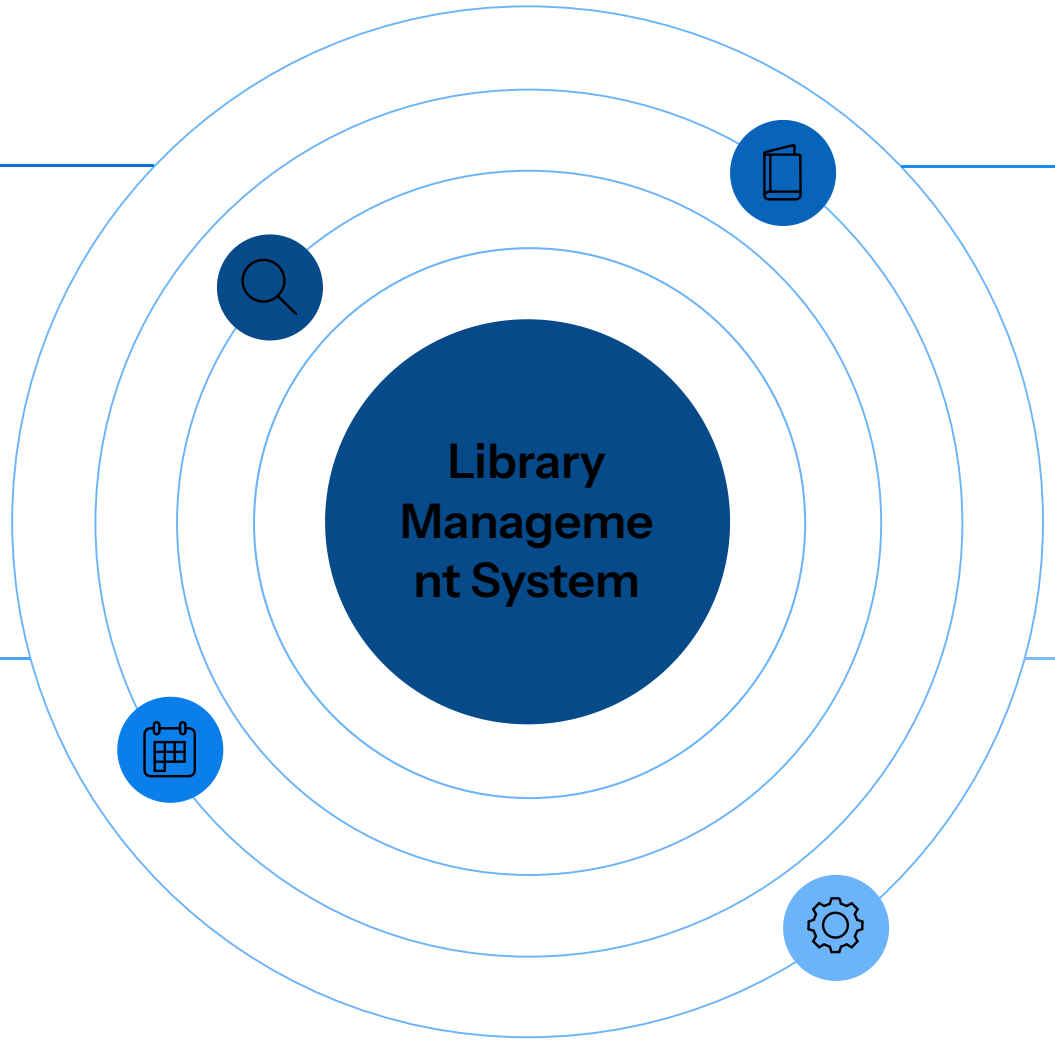
Structured Chart



Structured chart shows module decomposition: Presentation → Controllers → Services (CatalogueService, CirculationService, UserService) → Persistence. Use for modular design & responsibility assignment.

Made with GAMMA

Use Case Diagram — Actors & Primary Flows



Search Catalogue

Find books by title, author, or ISBN

Issue & Return

Checkout and checkin books for members

Reserve & Pay Fine

Place holds and process fine payments

Manage & Report

Inventory management and report generation

Caption: Use cases capture user goals and system responsibilities. Prioritise Search, Issue/Return and Reservation for MVP; Admin functions for maintenance and reporting.

1

High Priority

Search, Issue, Return, User Authentication

2

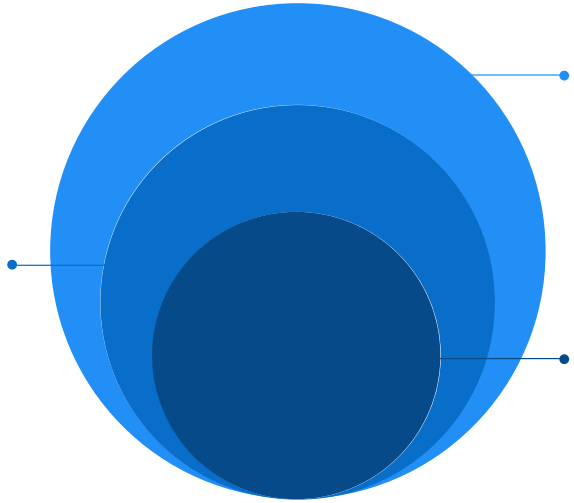
Secondary

Reservation, Fines, Notifications

Made with GAMMA

Class Diagram & Object Diagram

Class diagram outlines main entities and associations: Book, Copy, Member, Loan, Reservation, UserAccount, Fine, Inventory, Supplier. Emphasise composition between Book and Copy, aggregation between Member and Loan.



Interactions

Member → Loan (loanID, dueDate); Loan references Copy

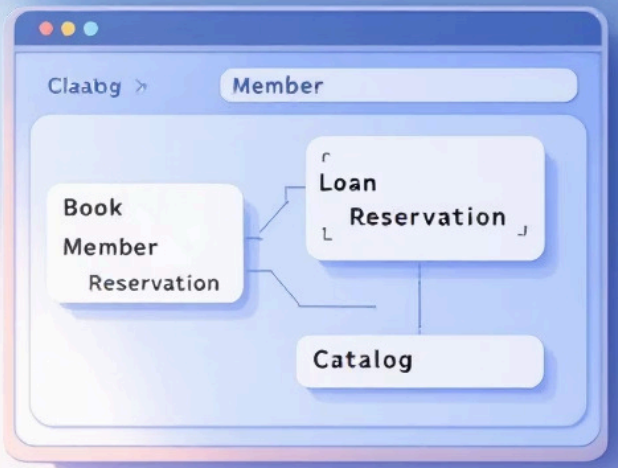
Contained Entities

Copy (copyID, status) — composition

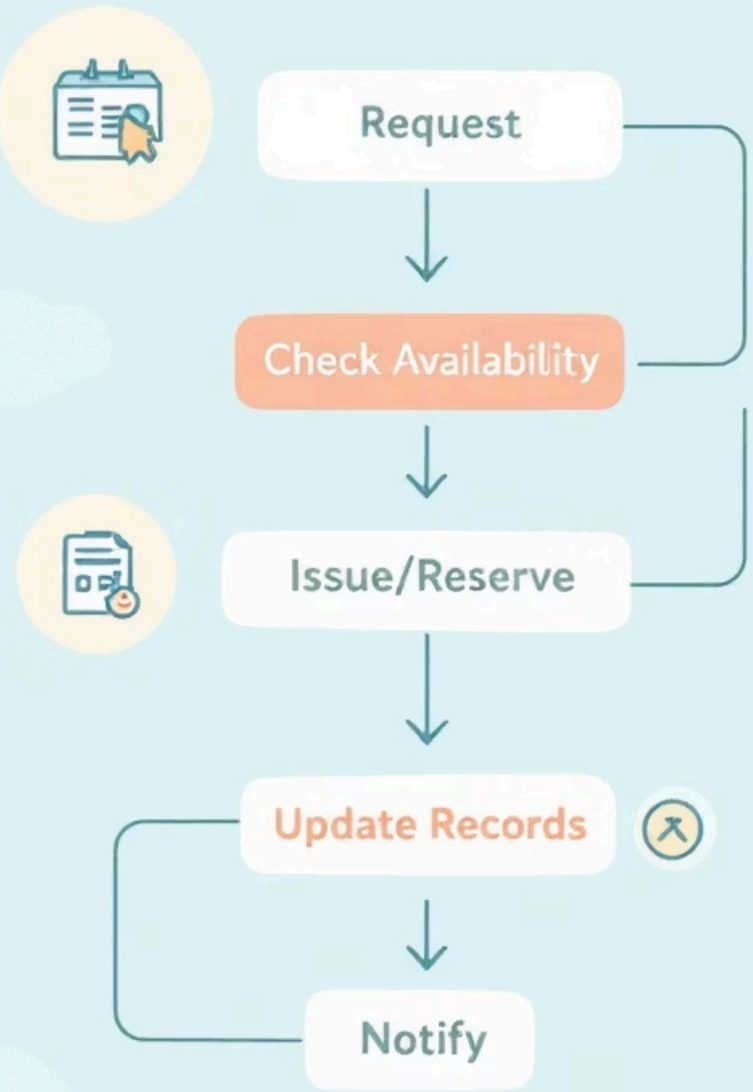
Core Model

Book (ISBN, title)

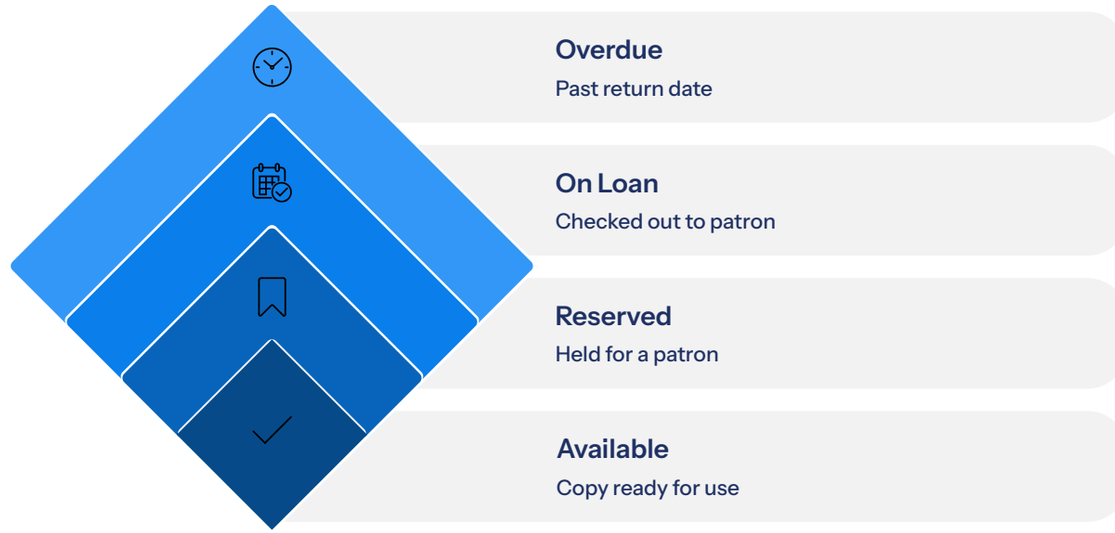
Object Diagram (example snapshot): shows specific instances — Book: 'Data Structures', Copy: #23, Member: M102, Loan: L9001 (issued).



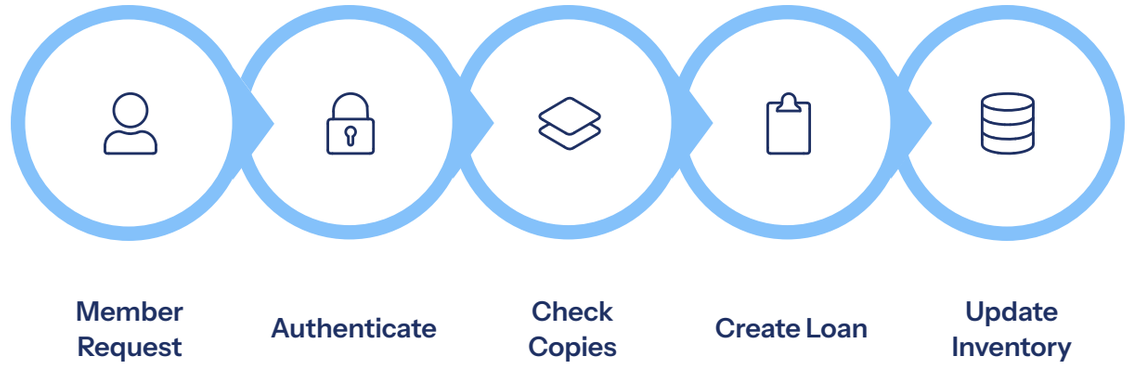
Made with GAMMA



State-chart & Activity Diagrams

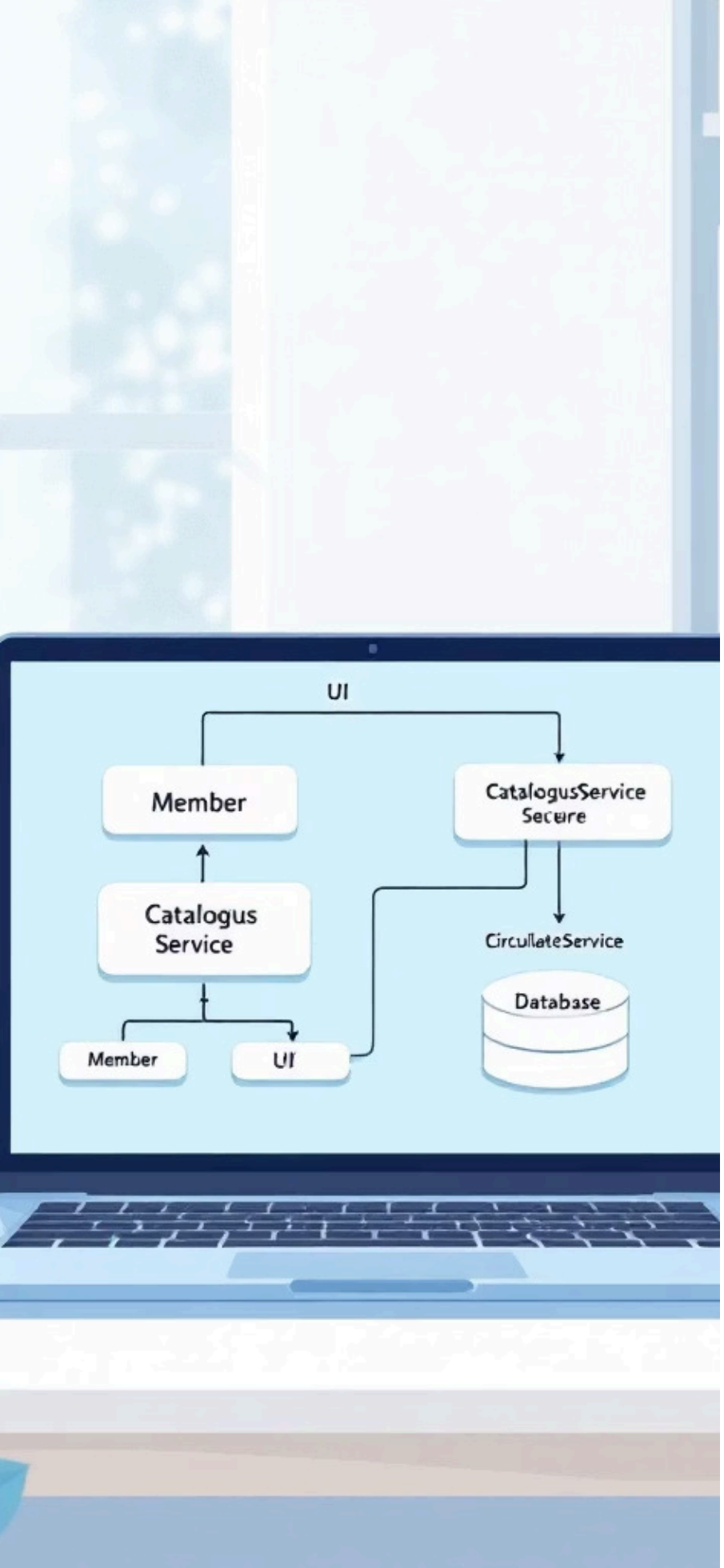


Caption: State-chart tracks lifecycle of a physical copy. Use it to drive UI states and validation rules.



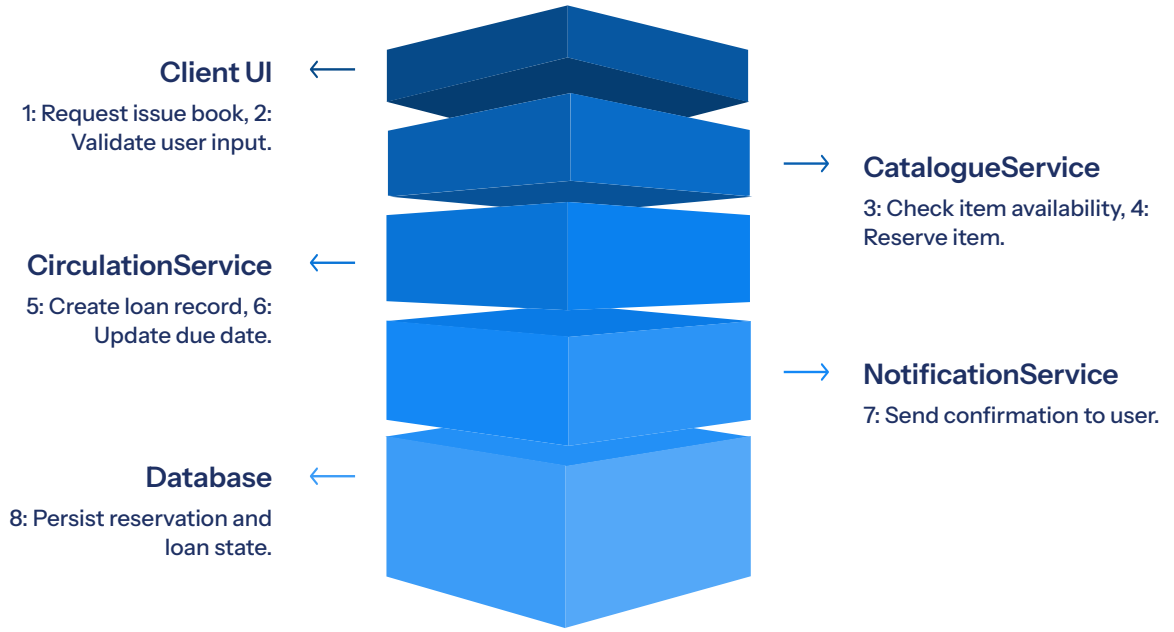
Caption: Activity diagram emphasises parallel actions (update inventory and send notification) and decision points (available vs not available).

Made with GAMMA



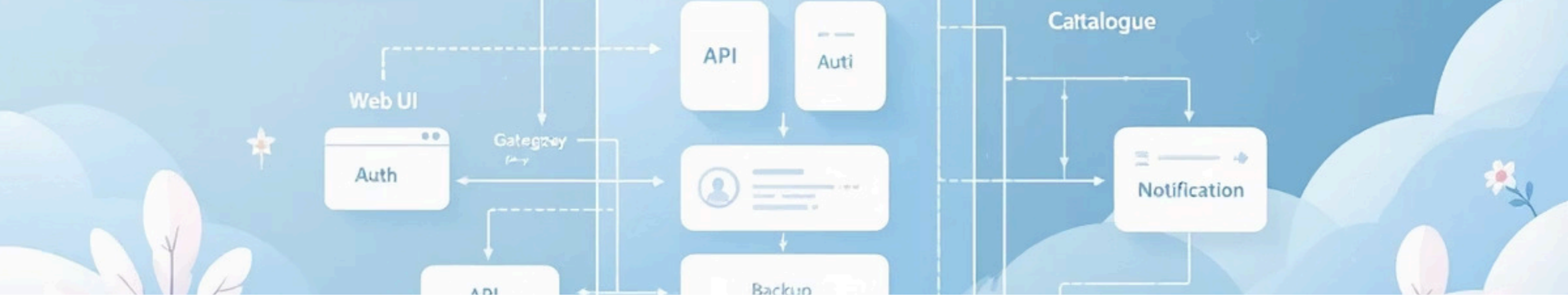
Sequence & Collaboration Diagrams

Caption: Sequence diagram details messages and timing between objects during issue flow. Useful for API design and transaction boundaries.

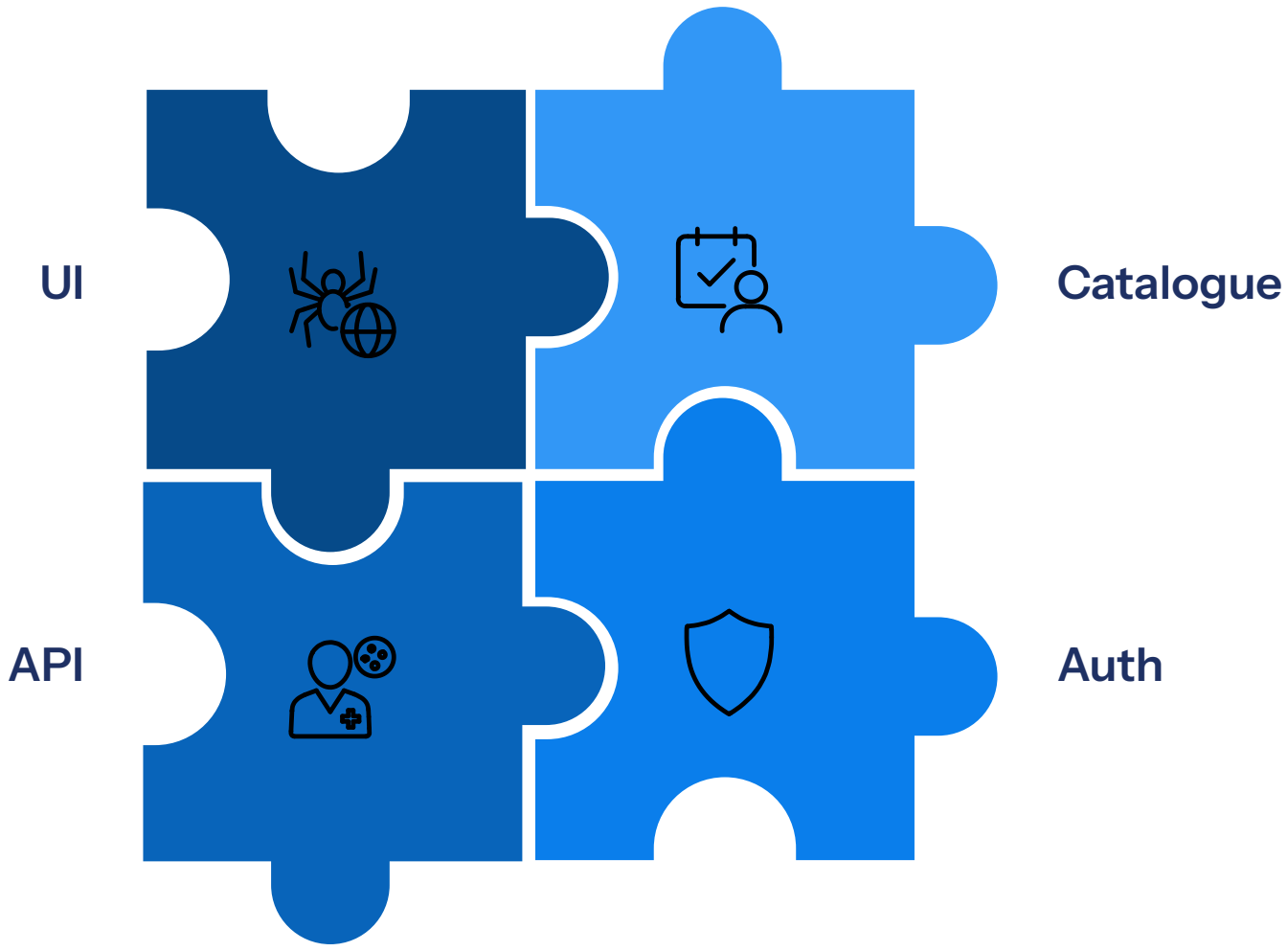


Caption: Collaboration view shows object links and responsibilities — valuable when designing component interactions and concurrency handling.

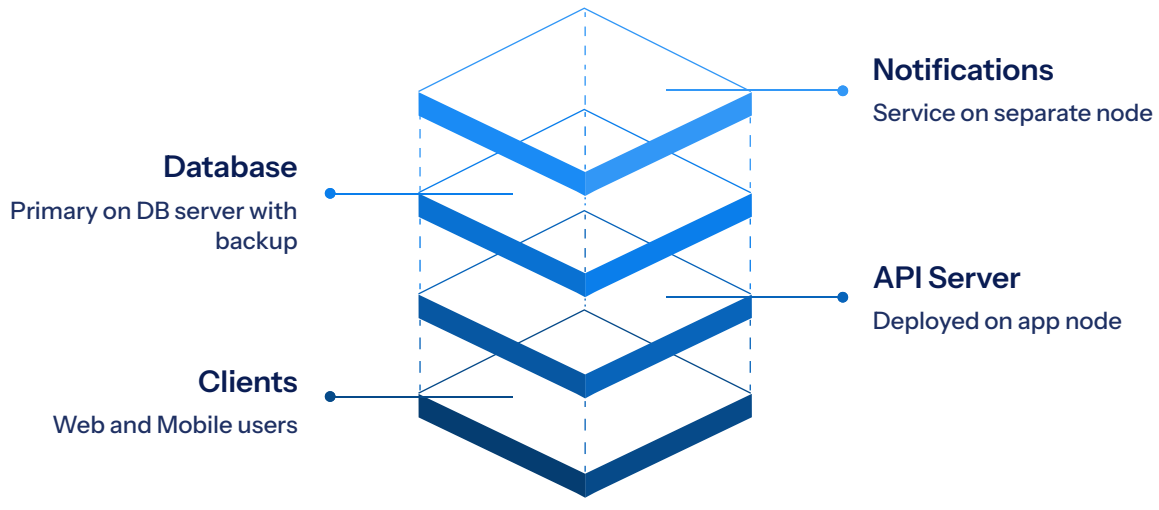
Made with GAMMA



Component & Deployment Diagrams



Caption: Components define code boundaries for teams; design microservices around Catalogue and Circulation for scalability.



Deployment notes: Use separate DB for read replicas, enable secure communication (TLS), containerise services for portability, and include offline kiosk mode for intermittent networks.

Made with GAMMA

Project Estimation — Function Point (FP) Method

Brief FP steps: identify functional components (EI, EO, EQ, ILF, EIF), classify complexity, compute Unadjusted Function Points (UFP), apply Value Adjustment Factor (VAF) to get Adjusted FP, convert to effort using productivity factor.

1 Step 1: Identify

EI: Search, Issue; EO: Notifications; ILF: Catalogue, Member DB; EIF: External Payment Gateway

2 Step 2: Count

Example UFP ≈ 160 (sample counts). VAF = 0.85 → Adjusted FP ≈ 136

3 Step 3: Effort

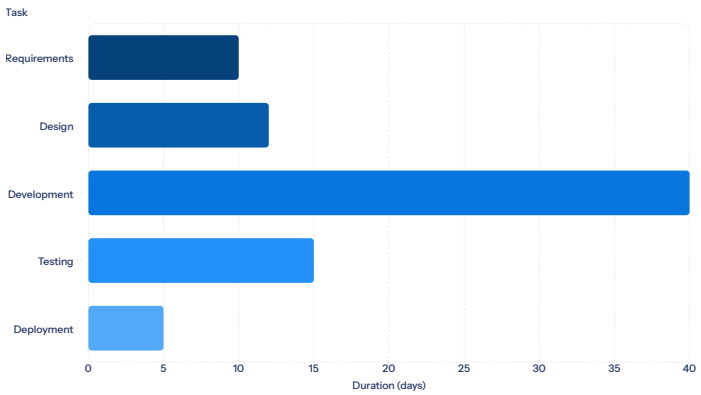
Conversion: 15 FP/person-month → ~9 person-months for core system (adjust per organisation)

Use FP for early budgeting, then refine with LOC or story-point estimates during planning.

Made with GAMMA

Schedule & Testing

Gantt / PERT Overview



PERT note: Identify critical path (Design → Development → Testing). Add 10% contingency for integration risks.

Test Cases Summary

Unit Tests

Validate CatalogueService methods, Loan calculation, fine computation. Edge cases: zero-availability, date boundary.

Integration Tests

Catalogue ↔️ Circulation ↔️ Notification integration; DB transaction rollback and concurrency tests.

White Box

Path coverage for fine calculation, authentication flows, and data validation routines.

Black Box

Functional acceptance: Search, Issue/Return, Reservation, UI workflows and accessibility checks.

Prioritise regression suites after each release. Target >=80% automated coverage on critical modules.

Next steps: review diagrams with stakeholders, finalise FP numbers with domain expert, begin iterative sprints with continuous testing and deployment pipelines.

Made with GAMMA